If your application is utilizing the Mailgun driver, you may find additional information about tags and metadata in Mailgun's documentation. Similarly, the Postmark documentation may be reviewed for further information on their tag and metadata capabilities.

If your application sends emails over Amazon SES, you should utilize the **`metadata`** approach to attach SES tags to the message.

Custom Symfony Message

The **`MailMessage`** class's **`withSymfonyMessage`** function allows you to register a closure that will be executed with the Symfony Message instance before sending the message. This allows you to fully tweak the message before it is delivered:

```
use Symfony\Component\Mime\Email;
public function toMail(object $notifiable): MailMessage
{
 return (new MailMessage)
     ->withSymfonyMessage(function (Email $message) {
      $message->getHeaders()->addTextHeader(
       'Custom-Header', 'Header Value'
      );
     });
}
```

Broadcast Notifications

The **`broadcast`** channel uses Laravel's event broadcasting services to send alerts, allowing your JavaScript-powered frontend to receive them in real-time. If a notification allows broadcasting, a **`toBroadcast`** function on the notification class can be defined. This function takes a **`$notifiable`** entity and returns a **`BroadcastMessage`** instance. If the **`toBroadcast`** method does not exist, the data that should be broadcast will be gathered using the **`toArray`** method. The data received will be encoded as JSON and sent to your JavaScript-powered frontend. Let's look at an example of the Broadcast method:

```
use Illuminate\Notifications\Messages\BroadcastMessage;
```